BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Quiz 3
Semester: Spring 2025 Full Marks: 15

CSE 470: Software Engineering

Python

```python
class Message:
    def __init__(self):
        self.txt = ""  # message content
        self.rec = []  # list of recipients

    def send(self, text, recipients):
        self.txt = text
        self.rec = recipients

        for recipient in self.rec:
            # Check if recipient is in blocked list and handle retry logic
            retries = 3
            while retries > 0:
                if recipient in ["blocked@email.com", "spam@email.com"]:
                    print(f"Recipient {recipient} is blocked")
                    retries -= 1
                    if retries > 0:
                        print(f"Retrying... {retries} attempts left")
                        continue
                    else:
                        print("Maximum retries reached")
                        return
                break

            print(f"Sending to {recipient}")
            print(f"Message: {self.txt}")
            print("Message sent successfully")

    def add_attachment(self, file):
        print(f"Attaching file: {file}")

class NotificationService:
    def send_notification(self, user, notification):
        # Similar retry logic for notifications
        retries = 3
        while retries > 0:
                retries -= 1
                if retries > 0:
                    print(f"Retrying... {retries} attempts left")
                    continue
                else:
                    print("Maximum retries reached")
                    return
            break

        print(f"Notifying {user}")
```

```
47          print(f"Notification: {notification}")
48          print("Notification sent successfully")
49
50
51      class TextMessage(Message):
52          def __init__(self):
53              super().__init__()
54              self.char_limit = 160
55
56          def add_attachment(self, file):
57              # Text messages cannot have attachments
58              pass
59
60          def send(self, text, recipients):
61              if len(text) > self.char_limit:
62                  print(f"Text too long. Limit is {self.char_limit} characters")
63                  return
64              super().send(text, recipients)
65
66
67      class EmailMessage(Message):
68          def __init__(self):
69              super().__init__()
70              self.subject = ""
71              self.attachments = []
72
73          def add_attachment(self, file):
74              print(f"Attaching file: {file}")
75              self.attachments.append(file)
76
77          def send(self, text, recipients):
78              print(f"Subject: {self.subject}")
79              for attachment in self.attachments:
80                  print(f"With attachment: {attachment}")
81              super().send(text, recipients)
```

Java

```java
class Message {
   private String txt;  // message content
   private List<String> rec;  // list of recipients

   public Message() {
      rec = new ArrayList<>();
   }

   public void send(String text, List<String> recipients) {
      this.txt = text;
      this.rec = recipients;

      for (String recipient : rec) {
         // Check if recipient is in blocked list and handle retry logic
         int retries = 3;
         while (retries > 0) {
            if (recipient.equals("blocked@email.com") ||
               recipient.equals("spam@email.com")) {
               System.out.println("Recipient " + recipient + " is blocked");
               retries--;
               if (retries > 0) {
                  System.out.println("Retrying... " + retries + " attempts left");
                  continue;
               } else {
                  System.out.println("Maximum retries reached");
                  return;
               }
            }
            break;
         }

         System.out.println("Sending to " + recipient);
         System.out.println("Message: " + txt);
         System.out.println("Message sent successfully");
      }
   }

   public void addAttachment(String file) {
      System.out.println("Attaching file: " + file);
   }
}

class NotificationService {
   public void sendNotification(String user, String notification) {
      // Similar retry logic for notifications
      int retries = 3;
      while (retries > 0) {
            retries--;
            if (retries > 0) {
               System.out.println("Retrying... " + retries + " attempts left");
               continue;
            } else {
```

```
                System.out.println("Maximum retries reached");
                return;
            }
        }
        break;
    }
    System.out.println("Notifying " + user);
    System.out.println("Notification: " + notification);
    System.out.println("Notification sent successfully");
  }
}

class TextMessage extends Message {
  private final int charLimit = 160;

  public TextMessage() {
    super();
  }
  @Override
  public void addAttachment(String file) {
    // Text messages cannot have attachments
  }

  @Override
  public void send(String text, List<String> recipients) {
    if (text.length() > charLimit) {
      System.out.println("Text too long. Limit is " + charLimit + " characters");
      return;
    }
    super.send(text, recipients);
  }
}

class EmailMessage extends Message {
  private String subject;
  private List<String> attachments;

  public EmailMessage() {
    super();
    attachments = new ArrayList<>();
  }

  @Override
  public void addAttachment(String file) {
    System.out.println("Attaching file: " + file);
    attachments.add(file);
  }

  @Override
  public void send(String text, List<String> recipients) {
    System.out.println("Subject: " + subject);
    for (String attachment : attachments) {
      System.out.println("With attachment: " + attachment);
    }
    super.send(text, recipients);
  }
}
```

1.	**CO3**	a. Identify the code smells in the above implementation. Just write the line numbers where the code smell appears (e.g. Line 4-12: Comments, etc.) [7 Marks] b. Refactor the code to resolve all identified issues. Write only the corrected code. [8 Marks]	**15**

a

Line 3-4: Inappropriate Naming (`txt` and `rec` are unclear abbreviations)

Line 11: Comments (explanatory comment for retry logic that should be refactored into self-documenting code)

Line 13-23: Long Method (retry logic is complex and should be extracted)

Line 34: Comments (explanatory comment for retry logic that should be refactored)

Line 35-44: Duplicated Code (retry logic is duplicated from Message class)

Line 56-58: Refused Bequest (TextMessage inherits `add_attachment` but cannot use it)

b

```
class RetryService:
    def __init__(self):
        self.blocked_addresses = ["blocked@email.com", "spam@email.com"]
        self.max_retries = 3

    def try_operation(self, recipient, operation_name):
        retries = self.max_retries
        while retries > 0:
            if recipient in self.blocked_addresses:
                print(f"{operation_name} recipient {recipient} is blocked")
                retries -= 1
                if retries > 0:
                    print(f"Retrying... {retries} attempts left")
                    continue
                print("Maximum retries reached")
                return False
            return True
        return False

class Message:
    def __init__(self):
        self.content = ""
        self.recipients = []
        self.retry_service = RetryService()
```

```
    def send(self, text, recipients):
        self.content = text
        self.recipients = recipients

        for recipient in self.recipients:
            if self.retry_service.try_operation(recipient, "Message"):
                self.deliver_message(recipient)

    def deliver_message(self, recipient):
        print(f"Sending to {recipient}")
        print(f"Message: {self.content}")
        print("Message sent successfully")

class TextMessage(Message):
    def __init__(self):
        super().__init__()
        self.char_limit = 160

    def send(self, text, recipients):
        if self.validate_length(text):
            super().send(text, recipients)

    def validate_length(self, text):
        if len(text) > self.char_limit:
            print(f"Text too long. Limit is {self.char_limit} characters")
            return False
        return True

class EmailMessage(Message):
    def __init__(self):
        super().__init__()
        self.subject = ""
        self.attachments = []

    def add_attachment(self, file):
        print(f"Attaching file: {file}")
        self.attachments.append(file)

    def send(self, text, recipients):
        print(f"Subject: {self.subject}")
        for attachment in self.attachments:
            print(f"With attachment: {attachment}")
        super().send(text, recipients)

class NotificationService:
    def __init__(self):
        self.retry_service = RetryService()

    def send_notification(self, user, notification):
```

```
if self.retry_service.try_operation(user, "Notification"):
    print(f"Notifying {user}")
    print(f"Notification: {notification}")
    print("Notification sent successfully")
```